HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN: THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

ĐỀ TÀI: XÂY DỰNG WEBSITE ĐỌC TRUYỆN TRỰC TUYẾN METRUYEN

Sinh viên thực hiện: Nguyễn Thành Nam

Mã sinh viên: B23DCCN587

Nhóm: 11

Giảng viên hướng dẫn: Kim Ngọc Bách

Hà Nội - 2026

Mục Lục

1. Nội dung công việc đã thực hiện trong tuần 3

1.1. Khởi tạo Backend Environment 3

1.2. Cấu trúc dự án 3

2. Xây dựng code phần Backend 5

3. Kết nối Cơ sở dữ liệu (Database Connection) 5

4. Lập trình Module Xác thực (Authentication) 5

5. Kiểm thử với Postman 6

Test Case 1: Đăng ký tài khoản mới 6

Test Case 2: Đăng nhập 7

Test Case 3: Lấy thông tin user hiện tại 8

Test Case 4: Truy cập protected route không có token 9

6. Kết quả đạt được 10

7. Kế hoạch tuần tới 10

Tài liệu tham khảo 11

1. Nội dung công việc đã thực hiện trong tuần

Trong tuần thứ 3, em đã tiến hành khởi tạo dự án, xây dựng cấu trúc thư mục server và lập trình phần Backend (Server-side), tập trung vào việc kết nối cơ sở dữ liệu và xây dựng module xác thực người dùng(Đăng nhập/Đăng ký) sử dụng JWT và mã hóa mật khẩu bcrypt [1], [2]. Cụ thể:

1.1. Khởi tạo Backend Environment

- Công nghệ sử dụng:

- *Runtime:* Node.js
- *Framework:* Express.js [3]
- *Database:* MongoDB (Local + MongoDB Atlas)
- *ODM:* Mongoose [4]
- *Authentication:* jsonwebtoken, bcryptjs [1], [2]
- *Middleware:* express-validator, cors, dotenv [5], [6]

1.2. Cấu trúc dự án

-Tạo folder backend và khởi tạo dự án NodeJS với lệnh:

npm init -y

-Cài đặt các thư viện lõi (Dependencies) với lệnh:

npm install express mongoose dotenv cors bcryptjs jsonwebtoken

npm install -D nodemon

- *express:* Framework web cho Node.js
- *mongoose:* Thư viện ODM để làm việc với MongoDB
- *dotenv:* Quản lý biến môi trường bảo mật.
- *cors:* Cho phép frontend gọi API
- *bcryptjs:* Mã hóa mật khẩu
- *jsonwebtoken:* Tạo token xác thực
- *nodemon:* Auto restart server khi code thay đổi

-Backend được tổ chức theo kiến trúc phân lớp cho RESTful API, gần với MVC nhưng tách riêng tầng View sang frontend ReactJS. Do đó ở phía server chỉ tập trung vào Model, Controller, Route, Middleware và Utils.

Cấu trúc thư mục Backend dự tính

```
backend/
├── src/
│   ├── server.js              # Application entry point
│   ├── config/                # Configuration files
│   │   ├── database.js                # MongoDB connection setup
│   │   └── cloudinary.js              # Cloudinary service configuration
│   ├── controllers/           # Request handlers (business logic layer)
│   │   ├── authController.js          # Authentication logic (register/login/me)
│   │   ├── userController.js          # User CRUD logic
│   │   ├── categoryController.js      # Category CRUD logic
│   │   ├── comicController.js         # Comic CRUD logic
│   │   └── chapterController.js       # Chapter CRUD logic
│   ├── middlewares/           # Reusable middleware functions
│   │   ├── auth.js                    # JWT verification and role authorization
│   │   ├── errorHandler.js            # Centralized error handling middleware
│   │   └── upload.js                  # File upload middleware (Multer)
│   ├── models/                # Mongoose schema definitions
│   │   ├── User.js                    # User schema
│   │   ├── Comic.js                   # Comic schema
│   │   ├── Chapter.js                 # Chapter schema
│   │   ├── Payment.js                 # Payment schema
│   │   └── Category.js                # Category schema
│   ├── routes/                # API route definitions
│   │   ├── authRoutes.js              # Routes under /api/auth
│   │   ├── userRoutes.js              # Routes under /api/users
│   │   ├── comicRoutes.js             # Routes under /api/comics
│   │   ├── chapterRoutes.js           # Routes under /api/chapters
│   │   └── categoryRoutes.js          # Routes under /api/categories
│   └── utils/                 # Shared utility helpers
│       ├── jwt.js             # JWT helper functions
│       ├── asyncHandler.js    # Async wrapper for controller error handling
│       └── responseHelper.js  # Standardized API response formatter
├── .env                       # Environment variables
├── .gitignore
└── package.json               # Project metadata and npm scripts
```

-Trong kiến trúc này:

- `*models*/` quản lý cấu trúc dữ liệu và tương tác MongoDB.
- `*controllers*/` xử lý nghiệp vụ cho từng chức năng.
- `*routes*/` định nghĩa các endpoint API và điều hướng request tới controller.
- `*middlewares*/` xử lý xác thực, phân quyền, lỗi và upload file.
- `*config*/` chứa cấu hình hệ thống như kết nối MongoDB và Cloudinary.
- `*utils*/` chứa các hàm dùng chung như JWT helper, async wrapper và chuẩn hóa response.

- `*View*` không đặt trong backend vì giao diện người dùng được triển khai ở frontend bằng ReactJS.

2. Xây dựng code phần Backend

- **Tạo User Model** (models/User.js): Schema MongoDB cho bảng Users với các trường thông tin cơ bản, bảo mật, và middleware hash password trước khi lưu
- **Tạo Authentication Controller** (controllers/authController.js): Xử lý logic đăng ký (register), đăng nhập (login), và lấy thông tin user hiện tại (getMe)
- **Tạo JWT Utility** (utils/jwt.js) và cấu hình .env: Helper functions để tạo (generateToken) và verify JWT token, kèm cấu hình JWT_SECRET và JWT_EXPIRES
- **Tạo Authentication Middleware** (middlewares/auth.js): Middleware bảo vệ các protected routes (protect) và phân quyền theo role (restrictTo)
- **Tạo Auth Routes** (routes/authRoutes.js): Định nghĩa 3 routes: POST /register, POST /login (public), và GET /me (protected)
- **Cấu hình Database** (config/database.js): Thiết lập kết nối MongoDB sử dụng Mongoose và xử lý lỗi kết nối
- **Tạo Server Entry Point** (src/server.js): Khởi tạo Express app, kết nối database, cấu hình middleware (cors, json, urlencoded), đăng ký routes, và thiết lập error handler

3. Kết nối Cơ sở dữ liệu (Database Connection)

- Thiết lập kết nối thành công tới **MongoDB Atlas** (Cloud Database) thông qua Mongoose.
- Cấu hình xử lý lỗi kết nối (Error Handling) để đảm bảo server không bị crash khi mất kết nối DB.
- Hiện thực hóa Schema User (đã thiết kế ở Tuần 2) vào code Mongoose Model với các ràng buộc (validation) chặt chẽ.

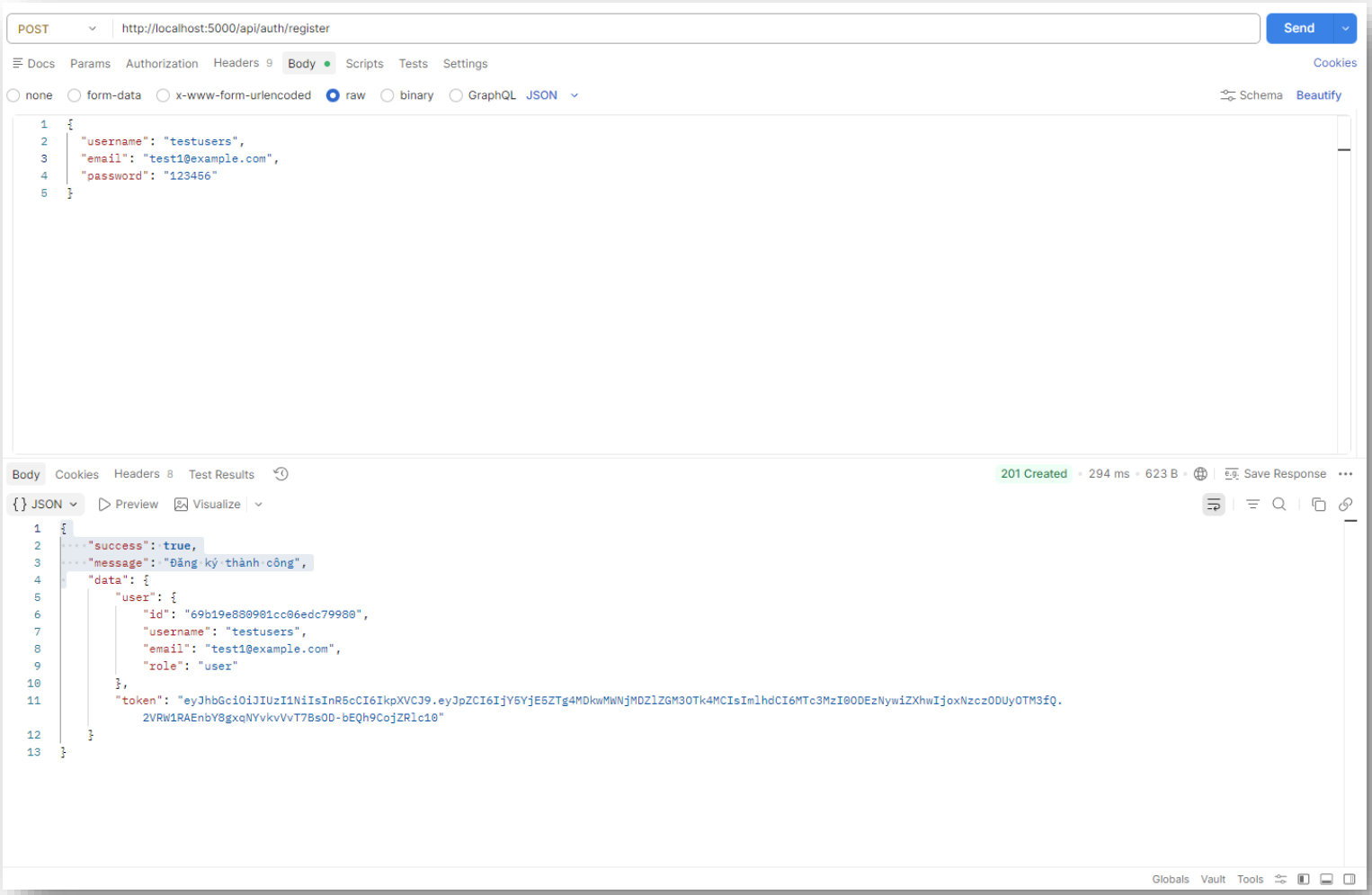
4. Lập trình Module Xác thực (Authentication)

- Cài đặt thư viện bảo mật:
  - bcryptjs: Dùng để băm (hash) mật khẩu một chiều trước khi lưu vào DB.
  - jsonwebtoken (JWT): Dùng để tạo và xác thực Token đăng nhập.

- Xây dựng 02 API cốt lõi:
    - **Đăng ký (Register):** Kiểm tra trùng lặp email/username, mã hóa mật khẩu và tạo tài khoản mới.
    - **Đăng nhập (Login):** Xác thực thông tin, cấp phát AccessToken (thời hạn 15 phút) và RefreshToken (thời hạn 7 ngày).
- Viết Middleware verifyToken để bảo vệ các Router yêu cầu quyền đăng nhập.

5. Kiểm thử với Postman

Test Case 1: Đăng ký tài khoản mới



=> Kết quả kiểm thử:

-Tạo mới tài khoản thành công, trả về success: true, thông tin user và JWT token;

-HTTP 201 Created

Test Case 2: Đăng nhập

POST http://localhost:5000/api/auth/login

Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "email": "test@example.com",
3   "password": "123456"
4 }
```

Body Cookies Headers 8 Test Results 200 OK 372 ms 644 B Save Response

{} JSON Preview Visualize

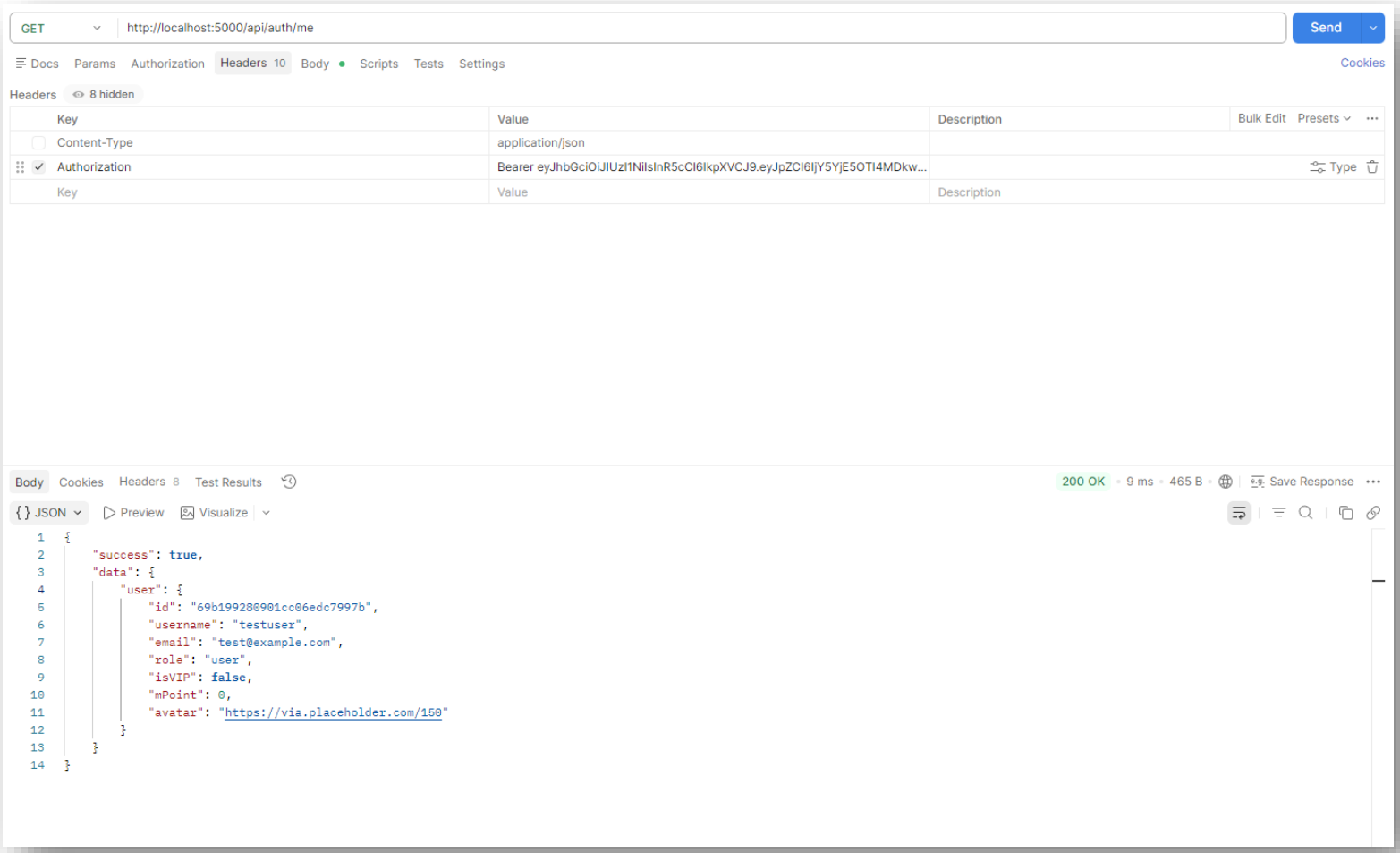

```
1  {
2      "success": true,
3      "message": "Đăng nhập thành công",
4      "data": {
5          "user": {
6              "id": "69b199280901cc06edc7997b",
7              "username": "testuser",
8              "email": "test@example.com",
9              "role": "user",
10             "isVIP": false,
11             "mPoint": 0
12         },
13         "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5YjE5OTI4MDkwMWNjMDZlZGM3OTk3YiIsImlhdCI6MTc3MzI0ODE4OCwiZXhwIjoxNzczODUyOTg4fQ.
               RbqE_wCPXN1qCYqm6VnWSEXcmR_A1oCjRjlptDeYlsI"
14     }
15 }
```


Globals Vault Tools

=> Kết quả kiểm thử:

- Đăng nhập đúng email/password, trả về success: true, thông tin user và JWT token;
- HTTP 200 OK

Test Case 3: Lấy thông tin user hiện tại

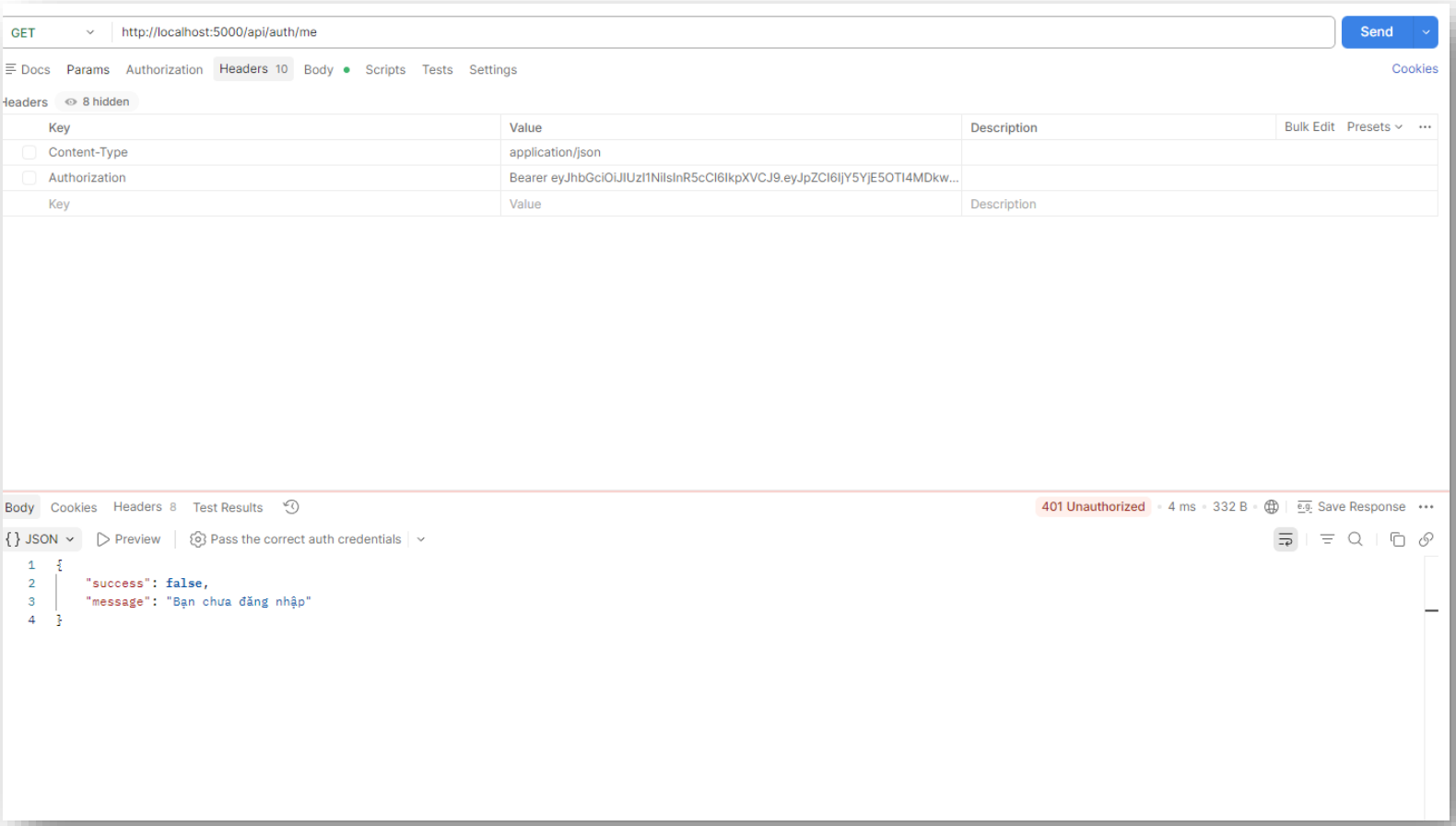


=> Kết quả kiểm thử:

- Khi gửi header *Authorization: Bearer <token>*, hệ thống trả về thông tin user hiện tại;

- HTTP 200 OK

Test Case 4: Truy cập protected route không có token



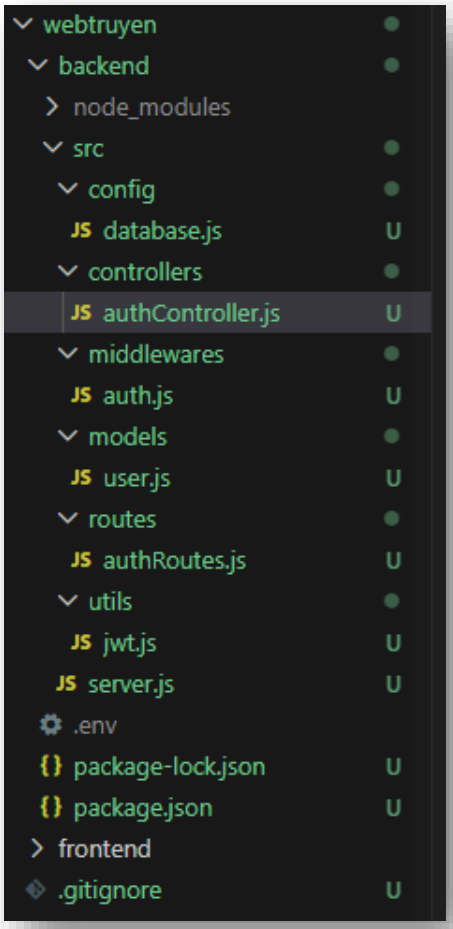
=> Kết quả kiểm thử:

- Không gửi Authorization header, hệ thống từ chối truy cập, trả về success: false, message Bạn chưa đăng nhập;

- HTTP 401 Unauthorized

6. Kết quả đạt được

- Khởi tạo dự án, xây dựng cấu trúc thư mục phần backend



- Server khởi chạy thành công trên cổng 5000.
- Kết nối ổn định với Mongo Compass và MongoDB Atlas.
- API Đăng ký và Đăng nhập hoạt động đúng logic (đã kiểm thử thành công bằng công cụ **Postman**).
    - *Kết quả test:* Tạo được User mới trong DB, mật khẩu đã được mã hóa, API Login trả về Token chuẩn JWT.

7. Kế hoạch tuần tới

- Xây dựng các API CRUD (Thêm, Sửa, Xóa, Xem) cho đối tượng Truyện (Comics) và Chương (Chapters).
- Tích hợp Cloudinary để xử lý chức năng Upload ảnh bìa truyện.
- Viết API lấy danh sách truyện có phân trang (Pagination) và lọc theo thể loại

Tài liệu tham khảo

[1] "JSON Web Token Introduction - jwt.io." Accessed: Mar. 07, 2026. [Online]. Available: https://www.jwt.io/introduction#what-is-json-web-token
[2] "bcryptjs," npm. Accessed: Mar. 12, 2026. [Online]. Available: https://www.npmjs.com/package/bcryptjs
[3] "Express - Node.js web application framework." Accessed: Mar. 12, 2026. [Online]. Available: https://expressjs.com/
[4] "Mongoose v9.3.0: Getting Started." Accessed: Mar. 12, 2026. [Online]. Available: https://mongoosejs.com/docs/
[5] "Cross-Origin Resource Sharing (CORS) - HTTP | MDN," MDN Web Docs. Accessed: Mar. 12, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS
[6] "dotenv," npm. Accessed: Mar. 12, 2026. [Online]. Available: https://www.npmjs.com/package/dotenv